PEAKNETFP: PEAK-BASED NEURAL AUDIO FINGERPRINTING ROBUST TO EXTREME TIME STRETCHING

Guillem Cortès-Sebastià[13] **Benjamin Martin**[2] **Emilio Molina**[1]
Xavier Serra[3] **Romain Hennequin**[2]
[1] BMAT Licensing S.L., Barcelona, Spain
[2] Deezer Research, Paris, France
[3] Music Technology Group, Universitat Pompeu Fabra, Barcelona, Spain
research@bmat.com, research@deezer.com

ABSTRACT

This work introduces *PeakNetFP*, the first neural audio fingerprinting (AFP) system designed specifically around spectral peaks. This novel system is designed to leverage the sparse spectral coordinates typically computed by traditional peak-based AFP methods. *PeakNetFP* performs hierarchical point feature extraction techniques similar to the computer vision model *PointNet++*, and is trained using contrastive learning like in the state-of-the-art deep learning AFP, *NeuralFP*. This combination allows *PeakNetFP* to outperform conventional AFP systems and achieves comparable performance to *NeuralFP* when handling challenging time-stretched audio data. In extensive evaluation, *PeakNetFP* maintains a Top-1 hit rate of over 90% for stretching factors ranging from 50% to 200%. Moreover, *PeakNetFP* offers significant efficiency advantages: compared to *NeuralFP*, it has 100 times fewer parameters and uses 11 times smaller input data. These features make *PeakNetFP* a lightweight and efficient solution for AFP tasks where time stretching is involved. Overall, this system represents a promising direction for future AFP technologies, as it successfully merges the lightweight nature of peak-based AFP with the adaptability and pattern recognition capabilities of neural network-based approaches, paving the way for more scalable and efficient solutions in the field.

1. INTRODUCTION

Audio Fingerprinting (AFP) is the MIR task of identifying audio recordings within a database of reference tracks. Early AFP systems date back twenty years, with Shazam [1] and Philips [2] systems. Since then AFP has been extensively studied for various use cases, such as query-by-example [1], integrity verification [3], content-based copy detection [4], DJ-set monitoring [5], or high specific audio retrieval [6]. Peak-based AFP systems have a long trajectory in the field and multiple works use this approach to enhance their robustness to pitch shifting and time stretching [7], background music identification [8], or to create a lightweight AFP that can run in embedded systems [9]. These algorithms are based on extracting and linking salient spectral peaks computed from time-frequency representations. These are mature, production-ready systems that do not require training, and can scale to industrial levels in which databases consist of millions of references [10,11]. Thus, companies with massive data catalogs rely on them for content identification [12].

Representation learning systems, such as *Now Playing* [13] or *NeuralFP* [6], recently emerge as novel approaches that leverage Contrastive Learning (CL) and Convolutional Neural Networks (CNNs) to learn the similarities between a distorted audio clip and its corresponding reference track. They are designed to perform highly sensitive audio retrieval, capable of matching short segments, and significantly outperform traditional peak-based methods under challenging conditions. This is due to their ability to capture more complex and nuanced features from the data, making them more robust to various types of distortions and noise that traditional methods struggle with [6, 14].

This comes at the expense of requiring large computational resources, dense input data, model training, and GPU computing, which might not be suitable for some applications. In industrial solutions, these requirements may be hard to overcome, and peak-based features are still considered as a viable alternative [11, 12, 15, 16]. Practically, it is common that audio features have to be computed on a client device, then uploaded to a server to perform identification against a reference database. In such conditions, requiring dense spectrograms as audio features significantly increases the amount of data to be uploaded compared to sending only sparse spectral peaks. When fingerprint generation is considered to run client-side, it may be more complex and battery intensive to run inference on trained models as compared to simpler rule-based peak extraction algorithms, especially for query-by-example applications [15] where client devices are generally variable in specifications (e.g. smartphones). An alternative setup is fully in-device audio identification [13], although this generally implies even more restrictive computational requirements

© G. Cortès-Sebastià, B. Martin, E. Molina, X. Serra, R. Hennequin. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** G. Cortès-Sebastià, B. Martin, E. Molina, X. Serra, R. Hennequin, "PeakNetFP: Peak-based Neural Audio Fingerprinting Robust to Extreme Time Stretching", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

on the device in terms of memory footprint and limits the database size. Additionally, music copyrights owners are often reluctant to share any dense representation that could be either inverted or used for other tasks than fingerprinting, and are more inclined to compute sparse targeted features that carry less information and can hardly be used for anything else than what they were designed to (with rare exceptions [17]). Finally, as peak-based AFP has been used extensively by industrial systems, it is relevant for private companies to leverage such large datasets of precomputed spectral peaks for neural audio fingerprinting approaches [1, 12]. For these reasons, in this work we propose to keep the traditional peak-based features as input, and use them in a modern neural approach.

In this first publication on a neural sparse peak-based model, we choose to focus our study on time stretching in extreme conditions, which has been underexplored in the literature. Time stretching is an audio processing technique that alters the tempo of a track without changing its pitch. This method is commonly used by DJs to synchronize the tempo of different songs within a mix or to create remixes that are either slowed down or sped up [18]. In challenging situations such as mash-ups, blends, or licensing circumvention attempts, time stretching happens in complex identification situations where severe tempo modifications are used on short excerpts, making them very hard to be automatically identified [19].

The main contribution of this work is to introduce a novel AFP system operating with lightweight spectral peaks as input, but grounded in a representation learning approach and evaluated in the context of time stretching. Specifically, our model *PeakNetFP* applies contrastive learning to learn fingerprints from sparse spectral peaks input, leveraging the hierarchical point set learning algorithm *Pointnet++* [20]. It is designed to exhibit the good performance of neural state-of-the-art approaches while keeping memory footprint low thanks to sparse input. To our knowledge, this is the first attempt at combining traditional peaks and representation learning for audio fingerprinting, and the first time a point-cloud network is used for AFP. As a subsequent contribution, we evaluate *PeakNetFP* alongside the SOTA algorithm on time stretching, *QuadFP* [21], which is a peak-based approach, and *NeuralFP* [6], the SOTA neural audio fingerprinting, in a new scenario to it. We finally show that *PeakNetFP* achieves performance close to the SOTA method *NeuralFP*, despite using 100 times fewer parameters and 11 times smaller input data than the latter.

In section 2 we summarize the works relevant to this publication. In section 3, we describe the hierarchical peak set feature extraction as well as the contrastive representation learning framework at the core of *PeakNetFP*. Finally, in section 4 we present its evaluation in the context of extreme time stretching and show how it compares to the peak-based time stretching baseline *QuadFP*, and to the spectrogram-based SOTA model *NeuralFP*. *PeakNetFP* code, dataset, and model are open and available [1] .

[1] https://github.com/guillemcortes/peaknetfp

2. RELATED WORK

Over the past two decades, the research community has worked to advance audio fingerprinting systems for multiple use cases. Some of these innovations include wavelets [22] for noise resilience, constant Q-transform [23] or Fundamental Frequency Map [24] for pitch-shifting robustness, and cosine filters [25] for broadcast monitoring, to name a few.

We can classify AFP methods into three broad categories: local descriptors-based [4, 22, 24–29], peak-based [1,7,19,21,23,28,30–32], and neural audio fingerprints [6, 13, 33–37]. Peak-based fingerprints started with Shazam's algorithm [1], which set the basis of spectral peak pairs linking to form hashes that are robust to noise. Then, Six & Leman proposed linking triplets to obtain robustness to time and frequency modifications in *Panako* [7], although it is not suitable for short queries or extreme time stretching since it was designed for content deduplication of audio collections of old recordings that were digitalized by replaying. Similarly, Sonnleitner & Wilder proposed *QuadFP* [21], which adapts blind astrometry research [38] to build quadruplets of peaks and generate hashes robust to significant time and frequency modifications [19]. Other peak-based AFP works [23, 28, 30–32] have also studied how to improve the robustness to time and frequency modifications. Peak-based traditional methods perform well even in the presence of alterations such as noise, compression, or reverberation, for instance. They generally produce lightweight hashes that can be efficiently indexed into lookup tables, which makes them scalable to hundreds of thousands or even tens of millions of music pieces. Additionally, such methods do not require training or accelerated computing hardware.

2.1 QuadFP

However, such traditional methods significantly underperform in the presence of extremely challenging scenarios, like in the case of strong time stretching [21]. *QuadFP* stands out as one of the most advanced peak-based AFP in that regard. Designed to be robust to time and frequency modifications, its core innovation is the use of quadruple peak descriptors, which capture not only the position of each peak but also its relationship with neighboring peaks. Each quadruple describes a constellation of four peaks (local maxima) in the time-frequency domain, effectively encoding local patterns and relationships between peaks. This approach is more robust to noise and variations in audio content compared to other fingerprinting methods [21] such as *Panako* [7]. Once the quadruple features are extracted, *QuadFP* uses a hashing mechanism to map these descriptors to a database. In this publication, we use *QuadFP* as the most advanced peak-based AFP baseline for robustness against time stretching. It also aligns with the use case of this study, which restricts the input data to spectral peaks. Our objective is to show how much a neural network can improve the best traditional system for time stretching.

2.2 NeuralFP

In the last decade, neural networks have been successfully used for AFP. In 2017, Google presented the first neural AFP system *Now Playing* [13]. It was designed to run on mobile devices with datasets of limited size while featuring high robustness to noise. A very recent approach, GraFPrint [39], leverages the structural learning capabilities of Graph Neural Networks (GNNs) to generate robust fingerprints from time-frequency representations. As opposed to our method, rather than using sparse spectral peaks, GraFPrint extracts from spectrograms localization-aware, low-dimensional features using a convolutional encoder. Other neural AFP systems [6, 33–37] use targeted augmentations in a contrastive learning framework to achieve robustness to noise, reverberation, echo, and other distortions. Among them, *NeuralFP* [6] stands out as being the only fully replicable neural AFP. The implementation is open-source, with a public dataset and model weights. *NeuralFP* is also lighter than other proposed models such as transformer-based AFPs [34, 36].

For these reasons, we use *NeuralFP* as a foundation for the development of *PeakNetFP*. *NeuralFP* leverages contrastive learning to achieve high-sensitive audio retrieval, employing a convolutional encoder to extract meaningful features from mel-spectrograms. In this paper, we propose to reuse *NeuralFP*'s contrastive learning framework while changing input data from dense spectrograms to sparse peak-based features. The original *NeuralFP* is then used as a reference model, showing what could be achieved when considering full spectrograms as input.

2.3 AFP for time stretching

Some fingerprinting methods have been designed to effectively handle time stretching, the focus of this work.

QuadFP [21] appears as a milestone on this topic. Using a quadruple-based spectral peak grouping (see section 2.1) combined with an asymmetric query-reference fingerprints configuration that maximizes the number of quads generated in queries, they report high precision and accuracy measures for multiple tempo modifications for 20 seconds queries. Their reference database consists of 100,000 tracks from Jamendo and they test 300 query tracks time-stretched with 13 stretching factors between 70% and 130%. In this experiment, they report an average accuracy of 92.9% for 10-second queries, but 28.1% average accuracy for 2.5-second queries. This shows how the performance collapses as the query length shrinks. We can expect this performance to be lower if fewer quads per second are used, a more likely scenario in an industrial environment.

Yao et al. [40] use the same dataset as [21]. Experiments were done for 13 stretching factors between 70% and 130% and a query length of 20 seconds. They report similar performance to *QuadFP* but with a 20% drop in recall. SAMAF [41] reports that for different query lengths ranging from 1 to 6 seconds, they achieve over 80% accuracy for mild stretching (0.9 and 1.1) but this collapses with severe stretching (0.5, 1.5), with less than 13% of accuracy. Panako [7] reports results for queries of 20, 40, and 60 seconds on a database of 30,000 songs. Less than a third of the queries are resolved correctly after a time stretching modification of 8%, though. Son et al. [24] achieve perfect precision for tempo modification in the range of 70% to 130%. However, their dataset only comprises 100 audio files and they query using the full audio length. George & Jhunjhunwala [42] propose to encode the features using only frequency information and thus making it independent of time, as opposed to [1], which encodes with respect to time. They test with tempo modifications in the ±50% range. They achieve over 97% of accuracy but on a small dataset of 300 samples of 20 seconds each. Their algorithm is also not suitable for short queries.

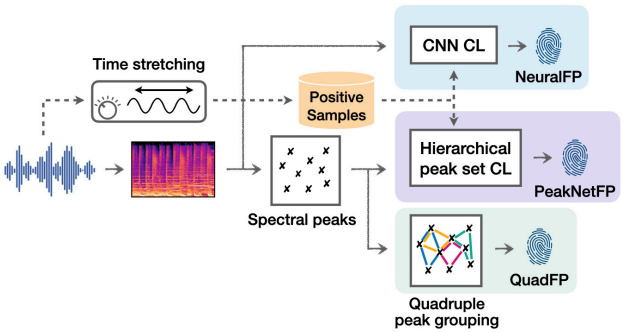


Figure 1. Considered AFP systems overview. From top to bottom: *NeuralFP*, *PeakNetFP* (ours), *QuadFP*. Dashed lines represent extra data used for training. Our model *PeakNetFP* learns features from the same input as *QuadFP*, in the same contrastive learning framework as *NeuralFP*.

3. PEAKNETFP

In this section, we describe our proposed model, *PeakNetFP*, starting from the sparse input data through to the contrastive learning framework, highlighting the hierarchical peak set feature extraction process. Additionally, we introduce the dataset used for evaluation. Figure 1 provides an overview of all the AFP systems considered, including our *PeakNetFP*, the baseline *QuadFP* [21], and the SOTA AFP model *NeuralFP* [6].

3.1 Sparse input data

As we described in the introduction, our model is designed to handle sparse data in the form of 3-dimensional spectral peaks, as features extracted from a third-party traditional AFP system. Typically, such peaks represent a subset of the local maxima from the spectrogram, chosen based on criteria that identify the most salient ones [1, 7, 30]. In our system, we extract local maxima in the melspectrogram using 3x3 kernels and stride 1 as a proxy for traditional peak-based fingerprints for simplicity and to avoid biasing the results on other system criteria. This allows the neural network to learn which peaks are most relevant for matching or classification even though a more refined peak selection could help reduce the dimensionality of the input,

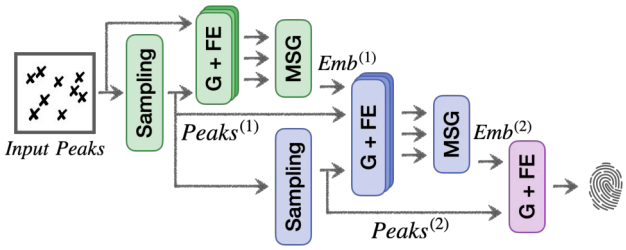


Figure 2. *PeakNetFP* overview consisting of Sampling, Grouping + Feature Extraction (G+FE) and Multi-Scale feature Grouping layers (MSG).

improving computational efficiency. In practice, we select the 256 highest amplitude local maxima per each 1-second segment, ensuring that we capture the most prominent features within each window.

When working with peaks rather than continuous data points, it becomes challenging to inject locality into convolutional kernels, which typically rely on dense, grid-like input structures. Peaks create a sparse representation of data, much like how point clouds are used in computer vision tasks. This sparsity complicates the direct application of traditional convolutional methods, as there is no inherent neighborhood structure. However, approaches like *PointNet* [43] and derived methods from the point cloud literature offer a potential solution by grouping local peaks and processing them similarly to how convolutions operate on spectrograms. By leveraging local relationships among peaks, we can capture meaningful patterns without requiring dense, continuous input.

3.2 Hierarchical peak set feature extraction

Hierarchical *PointNet*, or *PointNet++* [20], introduces a multi-level approach to capture both local and global features in sparse data, akin to the nested convolutions found in traditional CNNs. *PointNet++* organizes points (peaks in our context) into hierarchical groupings, where local neighborhoods are progressively sampled and processed, similar to the way convolutions scan across dense data. This hierarchical structure enables *PointNet++* to effectively learn both fine-grained and high-level features from sparse data. In *PeakNetFP*, we incorporate the hierarchical peak set feature extraction from *PointNet++* into the contrastive learning framework from *NeuralFP*. Figure 2 illustrates the architecture of *PeakNetFP* and Figure 3 the Grouping and Feature Extraction (G+FE) block of the second layer, represented in blue in Figure 2. The sparse peaks encoding starts with two Set Abstraction (SA) layers, represented in green and blue on Figure 2, which are responsible for grouping neighbouring peaks in a hierarchical way through Multi-Scale feature Grouping (MSG). Each layer i operates in three key steps:

(I) Sampling: The $N^{(i)}$ peaks with greatest amplitudes are selected as anchor peaks that will be the center of the peak groups. This step helps control computational complexity as the network goes deeper.

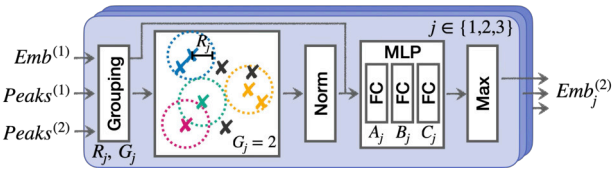


Figure 3. The Grouping + Feature Extraction block of the second layer ($i = 2$). For each specific sublayer j, R_j and G_j are the radius and group size of the queryball, and (A_j, B_j, C_j) are the dimensions of the MLP layers.

(II) Grouping + Feature Extraction block (G+FE): each block is made of 3 parallel layers, each layer j comprising:

(i) Grouping: for each anchor peak, we select the $G_j^{(i)}$ closest peaks withing radius $R_j^{(i)}$ using query balls to form local neighborhoods. These neighborhoods act as local receptive fields, similar to convolutional patches in CNNs. The query ball is a crucial element because it allows precise control over the distance and radius for hierarchical search in the point cloud. Unlike traditional convolutions that rely on fixed grid structures, the query ball adapts to the irregular distribution of peaks by grouping them based on actual spatial proximity.

(ii) Feature Extraction: Within each neighborhood, an MLP with 3 layers of respective dimensions $A_j^{(i)}, B_j^{(i)}, C_j^{(i)}$ is applied to learn local features. The MLP aggregates features for each point and uses max-pooling to summarize them into a single vector of dimension $N^{(i)} \times C_j^{(i)}$ representing the local region.

(III) Multi-scale feature grouping: Features of all parallel extraction layers are concatenated to a single embedding of dimension $N^{(i)} \times (C_1^{(i)} + C_2^{(i)} + C_3^{(i)})$. This step allows the model to capture features at multiple scales simultaneously by using different neighborhood radii during the grouping stage, which helps consider both fine and coarse features.

After each SA layer, the output is a smaller set of peaks with higher-dimensional feature vectors. These vectors are passed to the next SA layer, where the process repeats with a new sampling ($Peaks^{(2)}$ in Figure 2), further abstracting the data. As we move deeper into the network, the receptive fields become larger, allowing the network to capture broader contextual information while maintaining local details. The last SA layer, represented in purple in Figure 2, is similar to a Grouping + Feature Extraction block, but where *all* points are grouped together, forming a single 128-dimensional feature vector. This final vector thus encodes both local and global information about the peaks, and can be used as a fingerprint.

3.3 Contrastive learning framework

PeakNetFP relies on the *NeuralFP* contrastive learning framework [6] to learn fingerprints, which we describe in the following. It operates on 1-second windows with a 50% overlap. It creates data pairs by applying time stretching to short audio snippets. Each mini-batch MB

is formed by N samples and N augmented replicas of the same samples to generate positive pairs x_i and x_j so that $MB = \{x_i, x_j, ..., x_{Ni}, x_{Nj}\}$ and $|MB| = 2N$. NT-Xent loss [44] is chosen to maximize an agreement between positive pairs in a mini-batch MB. No explicit negative sampling is performed, thus, given a positive pair, the other $2(N-1)$ data points are to be treated as negative samples. The NT-Xent loss for a given pair of embeddings $\mathbf{z}_i$ and $\mathbf{z}_j$ is defined as:

$$l(i,j) = -\log \frac{\exp(a_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k\neq i]} \exp(a_{i,k}/\tau)} \quad (1)$$

Where $a_{i,j} = \mathbf{z}_i^T \mathbf{z}_j$ for $i, j \in \{1, \ldots, 2N\}$. τ is the temperature scaling factor for the softmax. Computing the Top-1 in the softmax function is equivalent to Maximum Inner Product Search (MIPS). $\mathbb{1}_{[k\neq i]}$ ensures that the summation excludes the anchor-positive pair. Then, the loss $\mathcal{L}$ averages l across all positive pairs, both (i, j) and (j, i):

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} l(2k-1, 2k) + l(2k, 2k-1) \quad (2)$$

During retrieval, as in [6], we retrieve 20 candidate segments from an Inverted File Product Quantization (IVFPQ) index built with *Faiss* [45]. Then, we perform a sequence matching in which each segment's embedding is compared to candidate embeddings via inner product, and results are ordered based on this score.

3.4 Dataset

To develop *PeakNetFP*, we use the same dataset as in *NeuralFP* [6] but change the augmentations to time stretching. The dataset consists in multiple audio files extracted from `fma_medium` dataset [46] that comes with defined subsets, which we also use to train and test our models. The *Train* subset contains 10,000 30-second audio clips while *Test-Query/DB* contains 500 30-second audio clips. To increase the reference set, we use *Test-Dummy-DB*, which comprises 100,000 full tracks with an average length of 278 seconds each. This is useful for testing the scalability of the system. In the evaluation step, we use the same 2,000 segments selected randomly from the 500 clips as in the *NeuralFP* evaluation.

During training, the stretching augmentations are performed at the spectrogram level, which we resize only on the time axis using bilinear interpolation. Unlike waveform-based stretching methods from `SOX`[2], this naive method integrates easily with the training pipeline and ensures that our model is not overfitting to any particularities of a specific model but learns to handle stretching. In testing, we use `SOX` to generate realistic queries from the DB tracks of *Test-Query/DB* set. We generate queries for stretching factors 1.05, 1.1, 1.2, 1.4, 1.6, 1.8, and 2 that increase the tempo of the song, and their counterparts reducing the tempo 0.975, 0.95, 0.9, 0.8, 0.7, 0.6, and 0.5. Note that a stretching factor of 2 doubles the tempo while 0.5 halves it. This test set is publicly available in Zenodo[3].

[2] `https://sourceforge.net/projects/sox/`

[3] `https://zenodo.org/records/15646861`

4. EVALUATION

In this section, we present the particularities of the evaluation framework as well as the metric used and the results. Finally, we also disseminate the computational cost of the benchmarked systems.

4.1 Evaluation framework

PeakNetFP evaluation is strictly based on *NeuralFP* to allow a fair comparison and can be examined in the repository accompanying this publication. We train both *PeakNetFP* and *NeuralFP* for 100 epochs using a batch size of 240 with Adam optimizer [47] following the authors' recommendation [6]. Table 1 summarizes the parameters of *PeakNetFP* layers, including the number of anchor peaks N, the queryballs radii R, and the number of peaks per queryball grouping G.

Since there is no public *QuadFP* implementation, we create our own version based on the original paper [21]. Although not all implementation details are provided for exact replication, we closely followed the model's key aspects, such as computing more quads for queries than references and applying cascading heuristics to efficiently filter out irrelevant quads during comparison. We validate our implementation in section 4.2 by comparing our results with the ones from the original publication [21].

We evaluate *PeakNetFP* as well as *QuadFP* and *NeuralFP* in the presence of time stretching ranging from the extreme values 0.5x to 2x the original speed. Additionally, we test each model with query lengths of 2, 3, 5, 6, and 10 seconds to ensure relevance for query-by-example applications. 1s queries are not considered, since their resulting size with time factors over 1 would make them smaller than *NeuralFP* window size.

To compare the AFP systems fairly, we align with the literature [6] by using Top-1 hit rate HR@1 defined as the number of hits at Top-1 divided by the number of queries. Note that both *NeuralFP* and *PeakNetFP* always returns a match, so in this case, Top-1 hit rate is equivalent to both precision and recall. Future work could include training a classifier on the matching scores to adapt the system to out-of-vocabulary queries.

					MLP		
Layer	N	j	G	R	A	B	C
SA + MSG 1	200	1	4	0.1	16	16	32
		2	8	0.2	32	32	64
		3	16	0.3	32	48	64
SA + MSG 2	100	1	4	0.2	32	32	64
		2	8	0.3	64	64	128
		3	16	0.4	64	64	128
SA					128	256	128

Table 1. *PeakNetFP* layers and their parameters: number of anchors N, layer index j, number of peaks to group G, grouping radius R, and dimensions of the 3 MLP layers A, B and C.

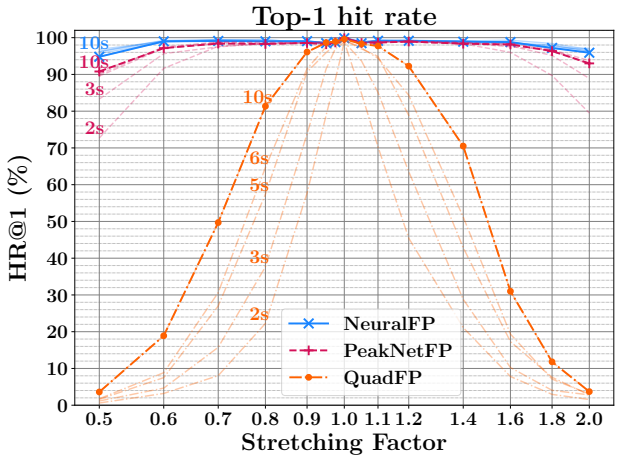


Figure 4. Top-1 hit rate (HR@1) as a function of stretching factors for *NeuralFP*, *PeakNetFP* (ours), and *QuadFP*. Each curve represents one (model, query length) pair.

4.2 Results

To check the validity of our custom implementation, we compare our *QuadFP* results with the ones reported on Table I in [21]. Our implementation of *QuadFP* shows better results than the original for short queries ($\leq$5 seconds), with an average 48% HR@1 for 2s queries in our case against a reported 28% for 2.5s queries in [21], and slightly worse results for larger ones, with 94% for our implementation vs 98% in [21] for 20-second queries. We acknowledge that the datasets differ between both studies (FMA versus Jamendo), but hypothesize that since their size and nature is similar, the comparison in the context of AFP systems still remains valid. In summary, we conclude that our implementation yields results comparable to [21], with remaining differences coming from either evaluation datasets or implementation differences.

Figure 4 illustrates Top-1 hit rates HR@1 of all 3 models as a function of stretching factor. For each model, we report 5 different curves that correspond to the 5 query lengths tested, drawn in blue plain lines for *NeuralFP*, orange dashed-dot lines for *QuadFP*, and magenta dashed lines for our model *PeakNetFP*. We highlight the curves corresponding to 10 seconds queries to compare with the best setup for our *QuadFP* baseline.

PeakNetFP outperforms *QuadFP* globally, effectively handling time stretching. It also exhibits excellent performance, achieving over 98% HR@1 for 10-second queries within the commonly reported 0.7 to 1.4 stretching factor range [21, 24, 40]. For extreme stretching factors (<0.7 and >1.4) *PeakNetFP* performance decreases a bit but still maintains over 90% HR@1. As a reference, *QuadFP* only achieves 3.6% HR@1 for a 0.5 factor. In fact, we observe that *QuadFP* performs well for minor stretching (0.9 to 1.1) as reported before [21], but its performance rapidly deteriorates as the stretching deviates further from 1, reaching nearly zero HR@1 at extreme stretching factors of 0.5 and 2. This effect is probably due to the lack of enough preserved quads at such strong time stretching factors. In terms of query length, *QuadFP*'s performance degrades quickly as queries get smaller. It should be reminded here that *QuadFP* is a rule based algorithm that does not require training as opposed to *PeakNetFP* or *NeuralFP*.

Results on the SOTA model *NeuralFP* exhibit a strong robustness to time stretching, even in extreme cases. As a reminder, *NeuralFP* processes entire spectrograms while *PeakNetFP* processes sparse peaks only. Nonetheless, for the commonly reported time stretching factors (0.7 to 1.4), *PeakNetFP* and *NeuralFP* obtain almost identical performance, with a maximum difference of $\pm$0.7% HR@1. For the more extreme factors, both systems performance is diminished, with *PeakNetFP* being more affected than *NeuralFP*, with the maximum difference between systems being 1.85% HR@1 at factor 0.5. Regarding the query length, *PeakNetFP* requires 5s queries at least to keep performance systematically above 0.9 at extreme time factors.

We conclude that *PeakNetFP* has a performance comparable to *NeuralFP* with a slight decrease at extreme stretching factors. However, *PeakNetFP* is significantly lighter than *NeuralFP*. It uses an input of 256 3D peaks, approximately 11 times smaller than *NeuralFP*'s 256×32 spectrograms. With 169k trainable parameters, *PeakNetFP*'s model size is 100 times smaller than *NeuralFP*'s 16.9M. This also requires substantially less inference memory: 800MiB for *PeakNetFP* versus 2338 MiB for *NeuralFP* (batch size 125 on a single RTX 3090). This improves our model's scalability and reduces memory usage for catalog embedding generation.

5. CONCLUSION

In this work, we introduce a novel audio fingerprinting system, *PeakNetFP*, that is designed as a hybrid approach combining the strengths of traditional peak-based fingerprint systems, heavily used in industrial contexts, with modern neural network-based representation learning approaches. We use a computer vision-inspired point cloud network to handle sparse peaks, which we use in a contrastive learning approach similar to modern AFP methods. Our evaluation in the context of extreme time stretching demonstrates that *PeakNetFP* consistently outperforms the SOTA on time-stretched data, *QuadFP*. Moreover, we show that, while the spectrogram-based SOTA in AFP *NeuralFP* performs very well in such a task, our model *PeakNetFP* can achieve comparable performance while working on peaks and thus requiring 11 times smaller input data, and using 100 times less parameters than the former.

In conclusion, *PeakNetFP* provides a scalable and efficient solution for audio identification tasks that involve significant tempo alterations, combining the compactness of peak-based methods with the robustness and flexibility of neural networks. It improves with respect to traditional methods for severe to extreme stretching factors, and appears as an alternative to fully neural approaches, especially for contexts where memory and computational efficiency are critical. Future works will focus on improving the model for applications beyond time stretching, such as pitch shifting.

6. ACKNOWLEDGEMENTS

This research is part of *resCUE – Smart system for automatic usage reporting of musical works in audiovisual productions (SAV-20221147)* funded by CDTI and the European Union - Next Generation EU, and supported by the Spanish Ministerio de Ciencia, Innovación y Universidades and the Ministerio para la Transformación Digital y de la Función Pública. Furthermore, it has received support from the Industrial Doctorates plan of the Secretaria d'Universitats i Recerca, Departament d'Empresa i Coneixement de la Generalitat de Catalunya, grant agreement No. DI46-2020.

7. REFERENCES

[1] A. Wang, "An industrial strength audio search algorithm," in *Proceedings of the 4th International Society for Music Information Retrieval Conference (ISMIR 2003)*, Baltimore, Maryland, USA, 2003, pp. 7–13.

[2] J. Haitsma and T. Kalker, "A highly robust audio fingerprinting system," in *Proceedings of the 3rd International Society for Music Information Retrieval Conference (ISMIR 2002)*, Paris, France, 2002, pp. 107–115.

[3] E. Gomez, P. Cano, L. Gomes, E. Batlle, and M. Bonnet, "Mixed watermarking-fingerprinting approach for integrity verification of audio recordings," in *Proceedings of the International Telecommunications Symposium*, Natal, Brazil, 2002.

[4] C. Ouali, P. Dumouchel, and V. Gupta, "A robust audio fingerprinting method for content-based copy detection," in *12th International Workshop on Content-Based Multimedia Indexing (CBMI 2014)*, Klagenfurt, Austria, 2014, pp. 1–6.

[5] R. Sonnleitner, A. Arzt, and G. Widmer, "Landmark-based audio fingerprinting for DJ mix monitoring," in *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR 2016)*, New York City, New York, USA, 2016, pp. 185–191.

[6] S. Chang, D. Lee, J. Park, H. Lim, K. Lee, K. Ko, and Y. Han, "Neural audio fingerprint for high-specific audio retrieval based on contrastive learning," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2021)*, Toronto, Ontario, Canada, 2021, pp. 3025–3029.

[7] J. Six and M. Leman, "Panako: A scalable acoustic fingerprinting system handling time-scale and pitch modification," in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR 2014)*, Taipei, Taiwan, 2014, pp. 259–264.

[8] H. Kim, J. Kim, J. Park, S. Kim, C. Park, and W. Yoo, "Background music monitoring framework and dataset for tv broadcast audio," *ETRI Journal*, 2024.

[9] J. Six, "Olaf: Overly lightweight acoustic fingerprinting," in *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR 2020)*, Montréal, Canada, 2020.

[10] A. L.-C. Wang and D. Culbert, "Robust and invariant audio pattern matching," United States Patent US007 627 477B2, 2009, Shazam Investments Ltd. and Apple Inc.

[11] A. Master, B. Mont-Reynaud, K. Mohajer, and T. Stonehocker, "Systems and methods for providing identification information in response to an audio segment," United States Patent US10 657 174B2, 2020, SoundHound, Inc.

[12] K. Akesbi, "Audio denoising for robust audio fingerprinting," Master's thesis, Ecole normale supérieure Paris-Saclay, Paris, France, 2022.

[13] B. Gfeller, B. Aguera-Arcas, D. Roblek, J. D. Lyon, J. J. Odell, K. Kilgour, M. Ritter, M. Sharifi, M. Velimirović, R. Guo, and S. Kumar, "Now playing: Continuous low-power music recognition," in *Proceedings of the 31st Annual Conference on Neural Information Processing Systems (NIPS 2017) Workshop: Machine Learning on the Phone*, Long Beach, CA, USA, 2017.

[14] G. Cortès, A. Ciurana, E. Molina, M. Miron, O. Meyers, J. Six, and X. Serra, "BAF: An audio fingerprinting dataset for broadcast monitoring," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR 2022)*, Bengaluru, India, 2022, pp. 908–916.

[15] A. Wang, "The shazam music recognition service," *Communications of the ACM*, vol. 49, no. 8, pp. 44–48, 2006.

[16] S. Bilobrov, "Indexing based on time-variant transforms of an audio signal's spectrogram," United States Patent US10 418 051B2, 2019, Facebook, Inc.

[17] M. Pfister, R. Michael, M. Boll, C. Körfer, K. Rieck, and D. Arp, "Listening between the bits: Privacy leaks in audio fingerprints," in *Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, Cham, 2024, pp. 184–204.

[18] D. Schwarz and D. Fourer, "Unmixdb: A dataset for dj-mix information retrieval," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR 2018)*, Paris, France, 2018.

[19] R. Sonnleitner, "Audio identification via fingerprinting. achieving robustness to severe signal modifications," PhD thesis, Johannes Kepler University Linz, Linz, Österrreich, 2017.

[20] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: deep hierarchical feature learning on point sets

in a metric space," in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS 2017)*, Long Beach, CA, USA, 2017, p. 5105–5114.

[21] R. Sonnleitner and G. Widmer, "Robust quad-based audio fingerprinting," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 3, pp. 409–421, 2016.

[22] S. Baluja and M. Covell, "Waveprint: Efficient wavelet-based audio fingerprinting," *Pattern Recognition*, vol. 41, no. 11, pp. 3467–3480, 2008.

[23] S. Fenet, G. Richard, and Y. Grenier, "A scalable audio fingerprint method with robustness to pitch-shifting," in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*, Miami, Florida, USA, 2011, pp. 121–126.

[24] H. Son, S. Byun, and S. Lee, "A robust audio fingerprinting using a new hashing method," *IEEE Access*, vol. 8, pp. 172 343–172 351, 2020.

[25] M. Ramona and G. Peeters, "Audioprint: An efficient audio fingerprint system based on a novel cost-less synchronization scheme," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2013)*. Vancouver, BC, Canada: IEEE, 2013, pp. 818–822.

[26] J. Haitsma, T. Kalker, and J. Oostveen, "Robust audio hashing for content identification," *Content Based Multimedia Indexing, Brescia, Italy*, 2001.

[27] X. Anguera, A. Garzon, and T. Adamek, "MASK: robust local features for audio fingerprinting," in *Proceedings of the 2012 IEEE International Conference on Multimedia and Expo, ICME*. Melbourne, Australia: IEEE Computer Society, 7 2012, pp. 455–460. [Online]. Available: https://doi.org/10.1109/ICME.2012.137

[28] E. Dupraz and G. Richard, "Robust frequency-based audio fingerprinting," in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP 2010)*, Dallas, Texas, USA, 2010, pp. 281–284.

[29] A. Agarwaal, P. Kanaujia, S. S. Roy, and S. Ghose, "Robust and lightweight audio fingerprint for automatic content recognition," 2023. [Online]. Available: https://arxiv.org/abs/2305.09559

[30] R. Sonnleitner and G. Widmer, "Quad-based audio fingerprinting robust to time and frequency scaling," in *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx-14)*, Erlangen, Germany, 9 2014, pp. 173–180.

[31] M. Malekesmaeili and R. K. Ward, "A local fingerprinting approach for audio copy detection," *Signal Process.*, vol. 98, pp. 308–321, 2014.

[32] J.-Y. Lee and H.-G. Kim, "Audio fingerprinting using a robust hash function based on the MCLT peak-pair," *The Journal of the Acoustical Society of Korea*, vol. 34, no. 2, pp. 157–162, 2015.

[33] Z. Yu, X. Du, B. Zhu, and Z. Ma, "Contrastive unsupervised learning for audio fingerprinting," *Computing Research Repository (CoRR)*, 2020.

[34] A. Singh, K. Demuynck, and V. Arora, "Attention-based audio embeddings for query-by-example," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR 2022)*, Bengaluru, India, 2022, pp. 52–58.

[35] X. Wu and H. Wang, "Asymmetric contrastive learning for audio fingerprinting," *IEEE Signal Process. Lett.*, vol. 29, pp. 1873–1877, 2022.

[36] A. Singh, K. Demuynck, and V. Arora, "Simultaneously learning robust audio embeddings and balanced hash codes for query-by-example," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2023)*, Rhodes Island, Greece, 2023, pp. 1–5.

[37] Y. Fujita and T. Komatsu, "Audio fingerprinting with holographic reduced representations," in *25th Annual Conference of the International Speech Communication Association (Interspeech 2024)*, Kos, Greece, 2024.

[38] D. Lang, D. W. Hogg, K. Mierle, M. Blanton, and S. Roweis, "Astrometry. net: Blind astrometric calibration of arbitrary astronomical images," *The astronomical journal*, vol. 139, no. 5, p. 1782, 2010.

[39] A. Bhattacharjee, S. Singh, and E. Benetos, "Grafprint: A gnn-based approach for audio identification," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2025)*, Hyderabad, India, 2025.

[40] S. Yao, B. Niu, and J. Liu, "Enhancing sampling and counting method for audio retrieval with time-stretch resistance," in *2018 IEEE Fourth International Conference on Multimedia Big Data (BigMM)*, 2018.

[41] A. Báez-Suárez, N. Shah, J. A. Nolazco-Flores, S.-H. S. Huang, O. Gnawali, and W. Shi, "SAMAF: Sequence-to-sequence autoencoder model for audio fingerprinting," *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, vol. 16, no. 2, 2020.

[42] J. George and A. Jhunjhunwala, "Scalable and robust audio fingerprinting method tolerable to time-stretching," in *2015 IEEE International conference on digital signal processing (DSP)*, 2015, pp. 436–440.

[43] C. R. Qi, H. Su, M. Kaichun, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification

and segmentation," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 77–85.

[44] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International Conference on Machine Learning (ICML), PmLR*, 2020, pp. 1597–1607.

[45] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[46] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR 2017)*, Suzhou, China, 2017, pp. 316–323.

[47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations (ICLR 2015)*, San Diego, CA, USA, 2015.